Jordan University of Science and Technology

Faculty of Computer and Information Technology

Department of Computer Engineering

CPE 591: Graduation Project I Report

Smart Blood Bank Management System

Mohammad Al-harahsheh 164280

Zaid Malkawi 165309

Abdallah Khwaileh 162197

Aws Al-khateeb 162925

Supervised by

Dr. Mohammad Al-shboul

Table of Contents

I. Abstract

Blood donation in Jordan can be considered disorganized, as there is no electronic system for the blood bank in Amman, nor any system linking hospitals to each other. This leads to several problems, including the lack of organization in collecting blood units during emergencies. Also, as mentioned, there is no electronic system; they rely entirely on paper records. This means that determining the number of blood units received or needed would be a lengthy process. Furthermore, there is no donor registry.

Our project will be a website and mobile application connecting blood donors, hospitals, and blood banks to provide blood units during emergencies and reduce donor search times using a database. Donors can register their contact information and blood type, hospitals can register patients, and the blood bank is responsible for monitoring blood stock levels. For example, if a hospital notifies the blood bank of a shortage of a specific blood type, the hospital will be supplied with the required units, and the blood bank will send notifications to donors and conduct donation drives to cover the shortage, making the blood donation process faster.

The most important feature of the project will be the cross country donation system, there has been cases where a person from Irbid for example went to an another governate where he doesn't have relatives or friends nearby and he would get into a car accident and there wouldn’t be donators from his social circle, this system would allow his friends from Irbid to donate to a bank near them and transfer their donation account to him.

Of course, this will cause the problem of one bank having too many donors with no one using these donations from the area, and the other bank will have a deficit in units. A solution is to make a transport vehicle for these units across banks to balance that deficit.

II. Background

- What is a Blood Bank

A blood bank is a center where blood gathered as a result of blood donation is stored and preserved for later use in blood transfusion. The term "blood bank" typically refers to a department of a hospital, usually within a clinical pathology laboratory, where the storage of blood products occurs and where pre-transfusion and blood compatibility testing is performed. However, it sometimes refers to a collection center, and some hospitals also perform collections. Blood banking includes tasks related to blood collection, processing, testing, separation, and storage. [7]

- Our motivation

In many countries, such as Saudi Arabia and the UK, there is a centralized digital application

meant to organize and manage the entire blood logistics lifecycle and donation process

from the initial donor to the final recipient.

  - After data collection by site visits to local blood bank centers and hospitals in Jordan, we conclude the following:

1. There is no dedicated digital system meant for managing the process from the donor's donation to the acceptor.

2. Documentation, management, and saving data are done through a physical handwritten file. This makes tracking the history and status (e.g., health, age, weight, blood type) of each donor and acceptor hard and complicated, thus requiring more effort and time to manage the whole process of donation.

By the data collected and our high motivation to develop this project, trying to overcome these gaps due to reliance on physical paper documentation and the weak community culture about the idea "giving blood", this system will provide an excellent secured framework to manage and deliver blood units with less time, less effort, and in a simple way.

- More about the Blood Bank Management System aims:

1. Alleviate Donor Hesitancy via a "Donor Credit" method:
   Donors may hesitate to donate blood without having a “Blood Credit” because some of the donor’s family or relatives, after a short while of donation, might indeed need blood, but the donated unit has been used for someone chosen randomly, and the donor now has to wait three to six months due to the needs for replacing lost blood cells and iron stores.

   To solve this problem and encourage the community to donate, the system will have a “Blood Credit”, where each donor can track their donations and give them to anyone easily, so that the donor can donate at any time freely without thinking about the amount of time indeed after each donation.

2. Improving the culture about giving blood and donation:
   With our system, we will help to create a community with a new point of view about blood donation and how it is easy and very helpful.

III. Proposed Work

• Introduction:

The Smart Blood Donation Management System is a platform designed to connect blood donors, hospitals, and blood banks in a unified system to improve the availability and management of blood supplies. The system aims to reduce the time required to find suitable donors and to minimize blood shortages during emergencies.

The platform will provide a centralized database where donors can register their blood type, contact information, location, and donation history. Hospitals and blood banks can monitor blood inventory levels and request specific blood types when shortages occur.

When a hospital reports a shortage, the system will automatically search for eligible donors with the required blood type and notify those who are located near the hospital or blood bank. This helps speed up the donation process and increases the chances of saving lives during critical situations.

That being said, our system will consist of mobile and web versions; this will provide high availability for all kinds of users.

In short, in Jordan, there are many Blood bank centers uses papers and old-school ways to manage blood donations. Our system will come to the rescue, not simply solve this problem, but also provide more services and technologies to take the blood donation process to another level. The system provides a full experience in the blood donation cycle to all kinds of users: hospital admins, Blood bank admins, Donors, and Patients.

One of the features that is really interesting is a bank-like system, where money here is blood, where you can transfer blood in a similar manner to transferring money, and the system will provide Blood balance management. There will be technical challenges; our system is designed to solve them.

- ## System Architecture:

The System has three distinct software layers using the **MVC** Architecture:

1. **The Model (Database Management & Pipeline Layer):** Manages the application's data, business rules, and state. It communicates directly with databases to retrieve, update, or save information.
2. **The Controller (The Core Service Layer):** The mediator and central brain between the Model and the View. It processes user inputs, requests the necessary data from the Model, and passes that data to the View to be rendered.
3. **The View (The User Interface (UI) layer):** Responsible for the presentation layer. It formats and displays the data received from the Model in a way that the user can see and interact with.

- ### Why MVC?

  Dividing the system into three main components will make collaboration much easier.
  Prevent tangled code, better maintenance, and scalability.

The system Architecture will consist of the following components; we will discuss every component and why we chose every technology in depth later on:

Frontend: Flutter[1] with Dart.

Backend: ASP.NET[2] Core with C#.

Database: PostgreSQL[3] .

Hosting and Deployment: Cloud service providers(AWS, Azure, Railway) + Docker[4] Containers.

Unit Testing: Different tools will be used to test the system's functionality.

- ## Backend :

The backend is the brain of our system; it will handle the following tasks:

3. **REST APIs**: The system leverages the client-server architecture using RESTful principles over HTTP/S. The backend handles incoming API requests from the Flutter applications and handles data exchange via JSON responses.
4. **Blood Balance Management**: To handle Blood Balance, you need to consider synchronization across the entire supply chain. From Donors donating their blood to someone or transferring it to another user to Hospitals using the blood units, the system will keep everything synchronized between all user platforms.
5. **CRUD Operations and Database Management**: This is for handling the database logic by providing a safe connection to our database and then executing the Create, Read, Update, Delete (CRUD) operations using the Entity Framework Core (EF Core) provided by the .NET Ecosystem.
6. **Security**: For handling user Authorization and Authentication, different technologies and libraries provided by the .NET framework will be used.

- ## Frontend:

To let users easily use our system, the frontend will be responsible for:

1. **Users Log in and sign up:** Delivers the User Interface components necessary to handle users log-in/sign-up flows using Flutter. This is the entry point for our system, sending the user credentials to the ASP.NET Core backend to validate them.
2. **Blood Balance and Personal Information:** Displays the user information, like Blood Balance and Personal info.
3. **Blood Donation Centers:** Show Blood Donation Centers on a live map, which will help the Donors to know what the best centers are to go to.
4. **Urgent Blood Units Notification System**: This will send to eligible users (Donors) the details of Emergency situations as notifications and guide the users to the best center for them.

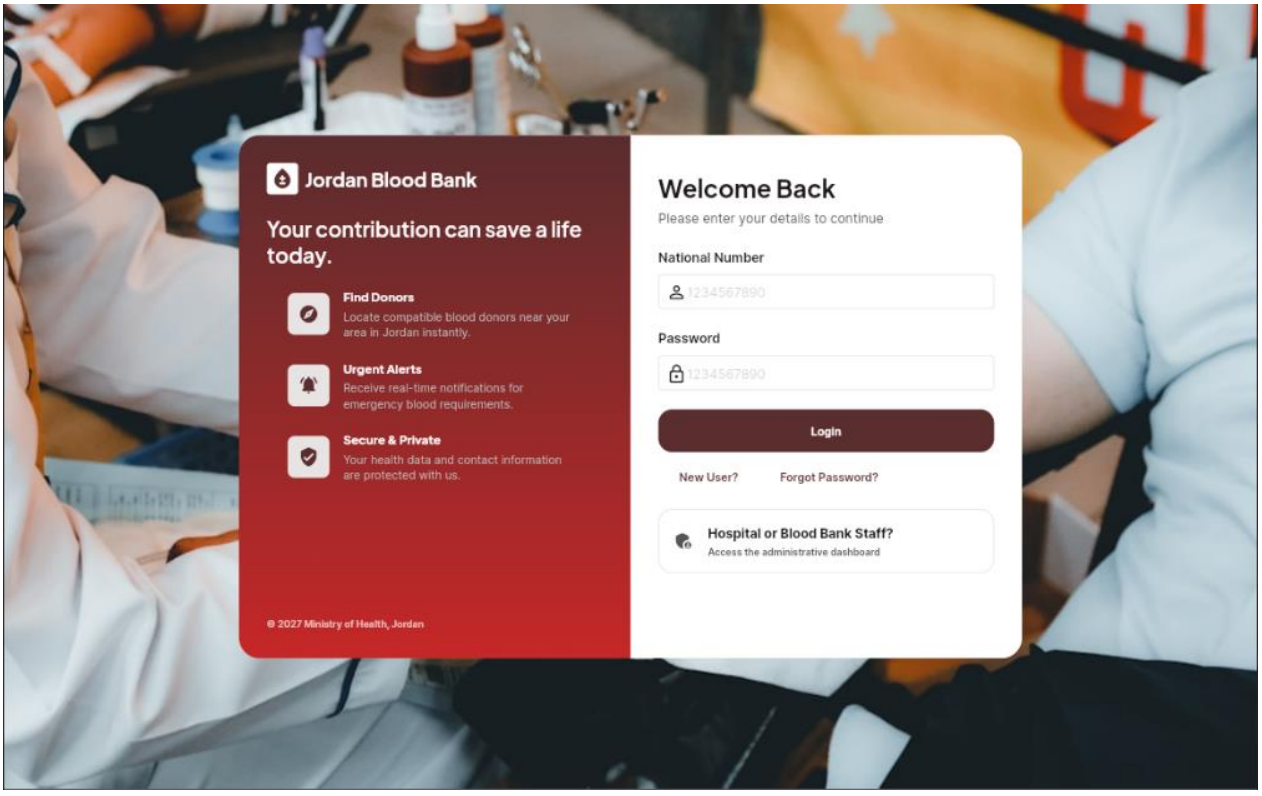

Figure 1: Login/Signup page

- As you can see in Figure 2, it's a demo of what the User Blood Wallet will look like.

- As you can see in Figure 3, it's a demo of how the admins will manage Shortage Alerts in the system.

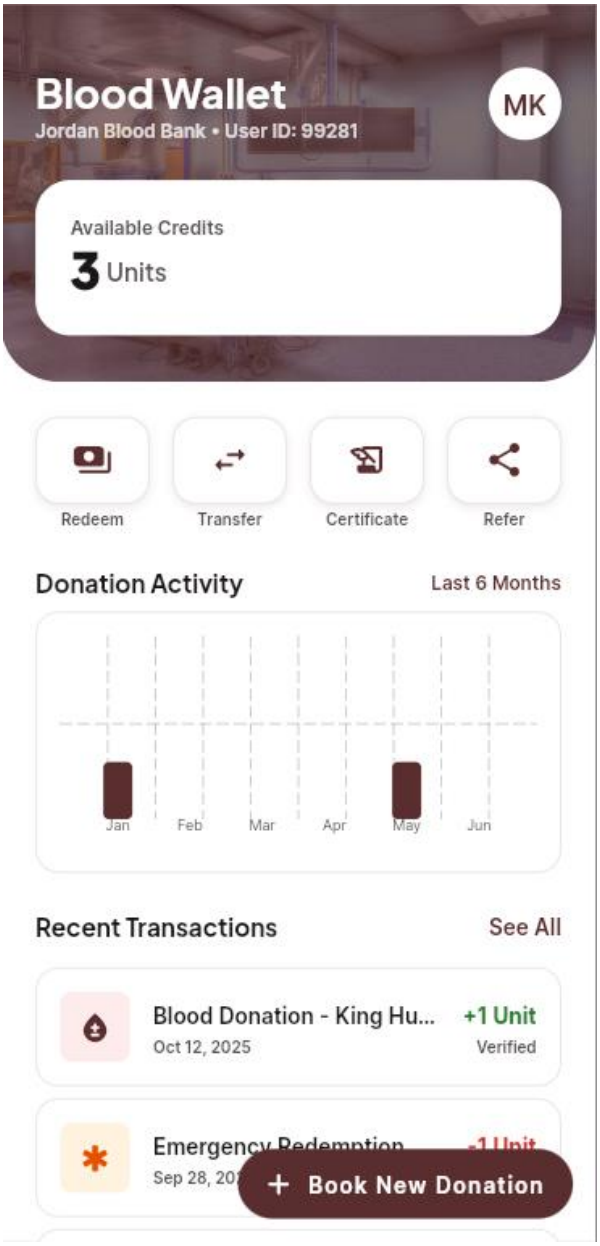

Figure 2: User Blood Wallet.

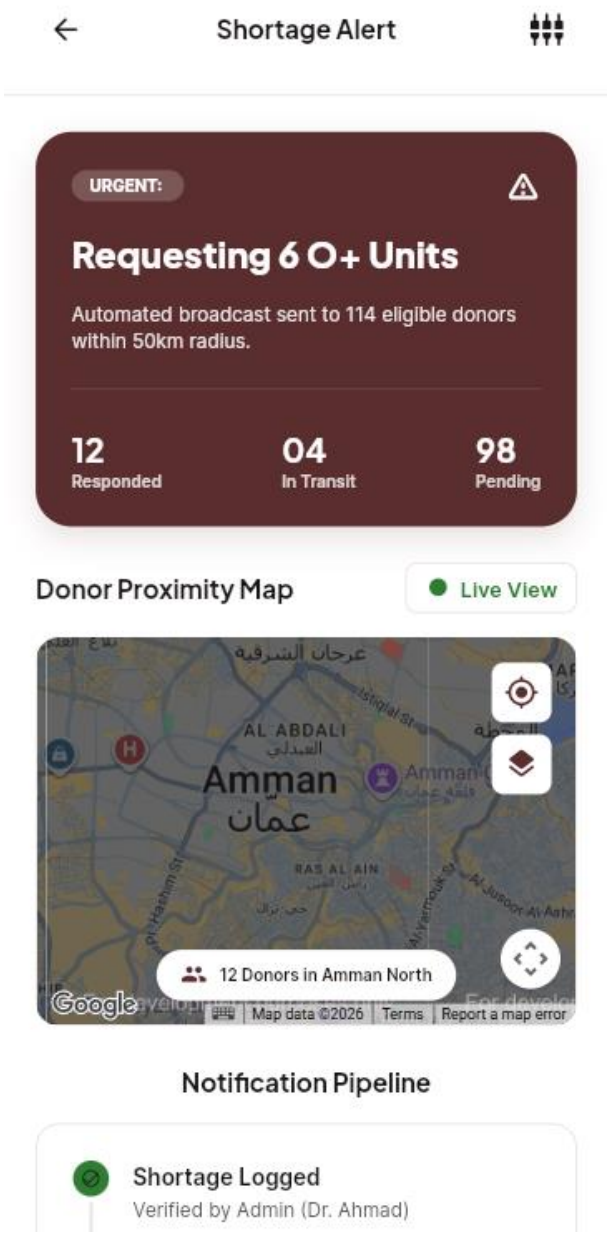

Figure 3: Shortage Alert Management for

- **Database:**

Here, all the data will be stored and secured. Some of the features it provides:

1. **Geospatial Location Data for Donation Centers:** This location data is structured and indexed to allow the backend to run fast spatial queries, instantly serving the correct coordinates required by the frontend live map.
2. **Blood Inventory:** Keeps structured records of all available blood units with their information, such as type, collection date, expiration date, and volume.

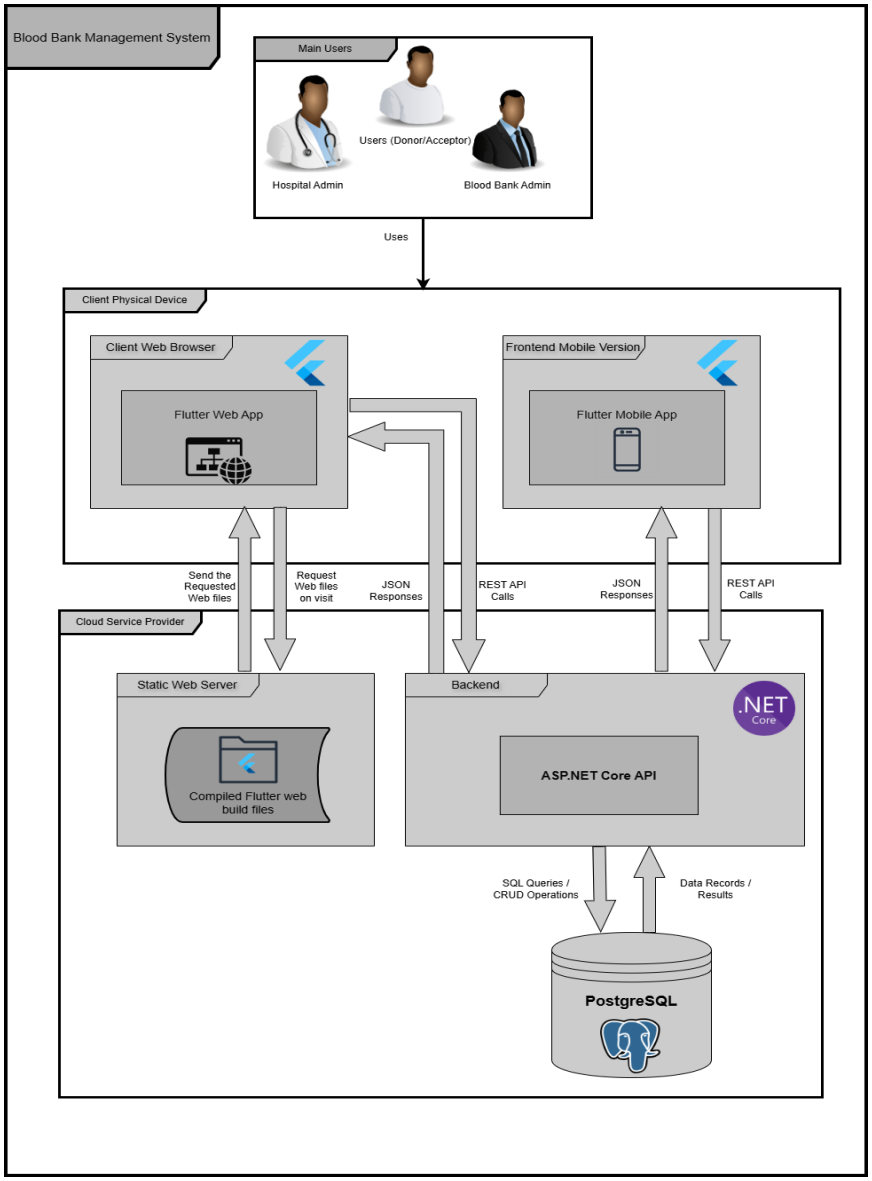

Figure 4: High-Level Architecture

- **Why this Tech Stack?**

For the **Backend,** using ASP.NET Core is a good choice because:

1. **Rich Ecosystem**: It provides, and it provides everything we need to build the system in one place.
2. **Built-in security**: Supports authentication, authorization, and data protection.
3. **Testing and Deployment**: It allows you to easily create unit and integration tests; it simplifies deployment with Docker containers, monitoring, and configuration.
4. **Productivity**: It's built to make the development process smoother and more productive.
5. **Scaling**: For our system, which is meant to serve a large number of users (50k+), choosing ASP.NET Core solves this challenge.

For the **Frontend**, Flutter is a great choice for several reasons:

1. **Cross-platform**: For our case, we don't have to write code for each platform (Mobile and Web); we just develop one codebase, and Flutter handles the rest.
2. **JavaScript Independent**: Unlike frameworks that rely on a JavaScript bridge, Flutter compiles directly into native ARM or Intel machine code, ensuring a smooth, responsive user experience.
3. **Hot Reload**: Lets developers make code changes and see them reflected on the screen almost instantly, without losing the app's current state.

For the **Database**, PostgreSQL is a great choice for several reasons:

1. **Security**: It supports a large number of Authentication technologies like GSSAPI, SSPI, LDAP, SCRAM-SHA-256, Certificate, and OAuth 2.0.
2. **ACID-compliant: ACID** indicates **A**tomicity, **C**onsistency, **I**solation, and **D**urability, and that means the data in a database is accurate because incomplete changes were never stored; in our case, that's really important because any tiny mistake could cause problems with the blood balance management system.
3. **Reliability, Disaster Recovery:** Ensures database reliability and disaster recovery through its core Write-Ahead Logging (WAL) engine, continuous physical backups, and built-in replication, which is important for our data to survive under any conditions.

Compatibility: How well can this tech stack work together?

The tech stack we are using is considered one of the most popular and widely used in the tech industry. Flutter communicates with the backend using standard HTTPS REST calls, then serializes Dart objects into JSON strings, where ASP.NET Core has a dedicated engine to accept and handle JSON strings. ASP.NET Core will

handle all API calls regardless of whether they are coming from the mobile or web. For ASP.NET Core to handle PostgreSQL, we have the **Npgsql** package; it gives you full compile-time safety and LINQ query support for PostgreSQL. That being said, our tech stack is highly compatible, and it has a large community that supports it.

- **Costs and Licensing:** To be noted, all the technologies we will be using are Open-Source projects, which means we will not worry about any licensing overhead.

Our system will consist of three primary Actors:

1- **Blood Bank Administrator**
Responsible for managing the entire platform, including donor records, hospitals, blood bank data, and monitoring blood inventory across the system.

2- **Hospital Administrator**
Each hospital will have its own administrator who can request blood units, update blood stock, confirm donor arrivals, and track donation records.

3- **Users (Donors/Acceptors)**
Users can register in the system, update their personal information, receive donation requests, and schedule blood donation appointments.

- **Functional Requirements:**

  - **User Authentication and Access Management**:
    1. The system will allow new users to create a new account by providing personal information like Name, National Number, Email, Date of Birth, phone number.
    2. The system will allow users to log in to their account by providing his/her National-Number/Phone Number/Email with their password.
    3. The system will enforce Multi-Factor Authentication (MFA) via SMS OTP and Email OTP upon login.
    4. The system should provide a secure password reset mechanism via verified contact methods.
    5. The system should distinguish between the Central Administrator, the Hospital Administrator, and Donors and restrict services accordingly.

  - **Account Management**:

1. The system should display an overview of the logged-in user (Personal photo, Full Name, Blood Type, Blood Balance).
2. The system must allow donors to view, filter, and download blood donation history.
3. The system must allow administrators to search and filter donors based on Name, Blood type, and location of residence.

- **Balance Transfer & Redeem**:

1. The system should allow Donors to use their accumulated balance to provide blood units for their relatives or friends during emergencies.
2. The system must allow users to redeem blood to their account by donating to a verified blood donation center.
3. The system must accumulate credits for a confirmed donation as a "Blood Credit" in the donor's digital wallet.

- **Hospital and Blood bank administration**:

1. The system must give hospital admins access to donors' credentials like Blood type, medical record (maybe through Hakeem), and Contact information.
2. The system must allow hospital admins to report blood shortages, and then the system will automatically search for eligible donors with the required blood type and notify those who are located near the hospital or blood bank.
3. The system must give admins a full record of the blood units' status in the hospital or blood bank.
4. The system must allow hospitals and blood banks to communicate with each other through a dedicated communication system.

- **Special Features**:

1. The system will provide detailed information about blood donation centers and present it on a map interface.
2. The system will manage blood transportation between cities or between hospitals/blood-banks.

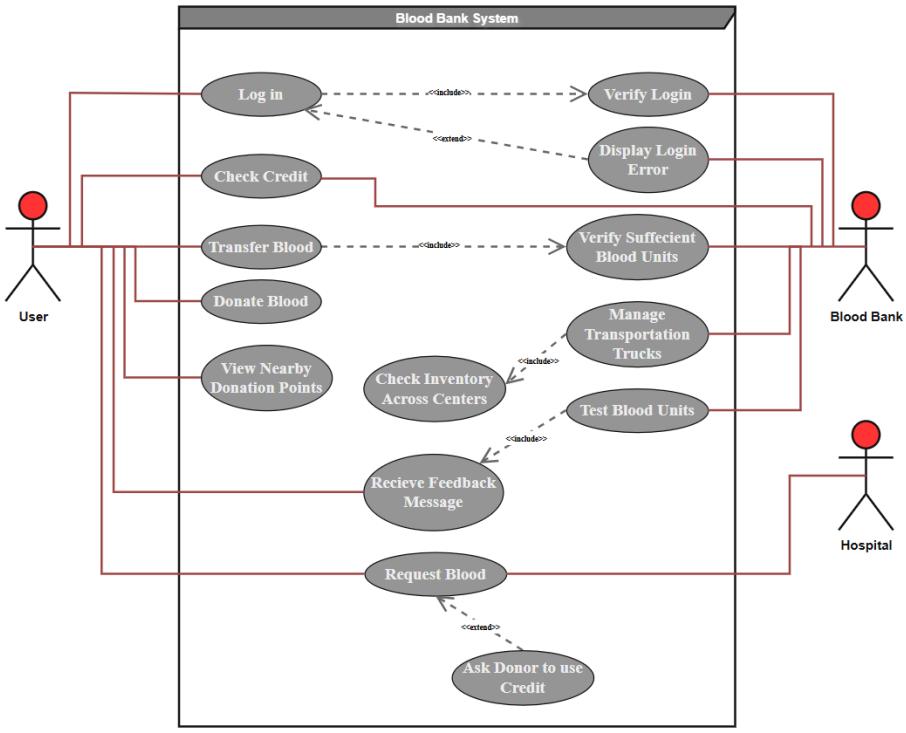

Figure 5: Use case diagram

Here is the use case diagram; it includes the three actors.

1. Primary actor “User” that could be a donor or an acceptor.
2. Blood Bank, which will validate and manage the user functions and the inventory.
3. Hospital, which can request blood units from the blood bank center or from a donor’s blood credit.

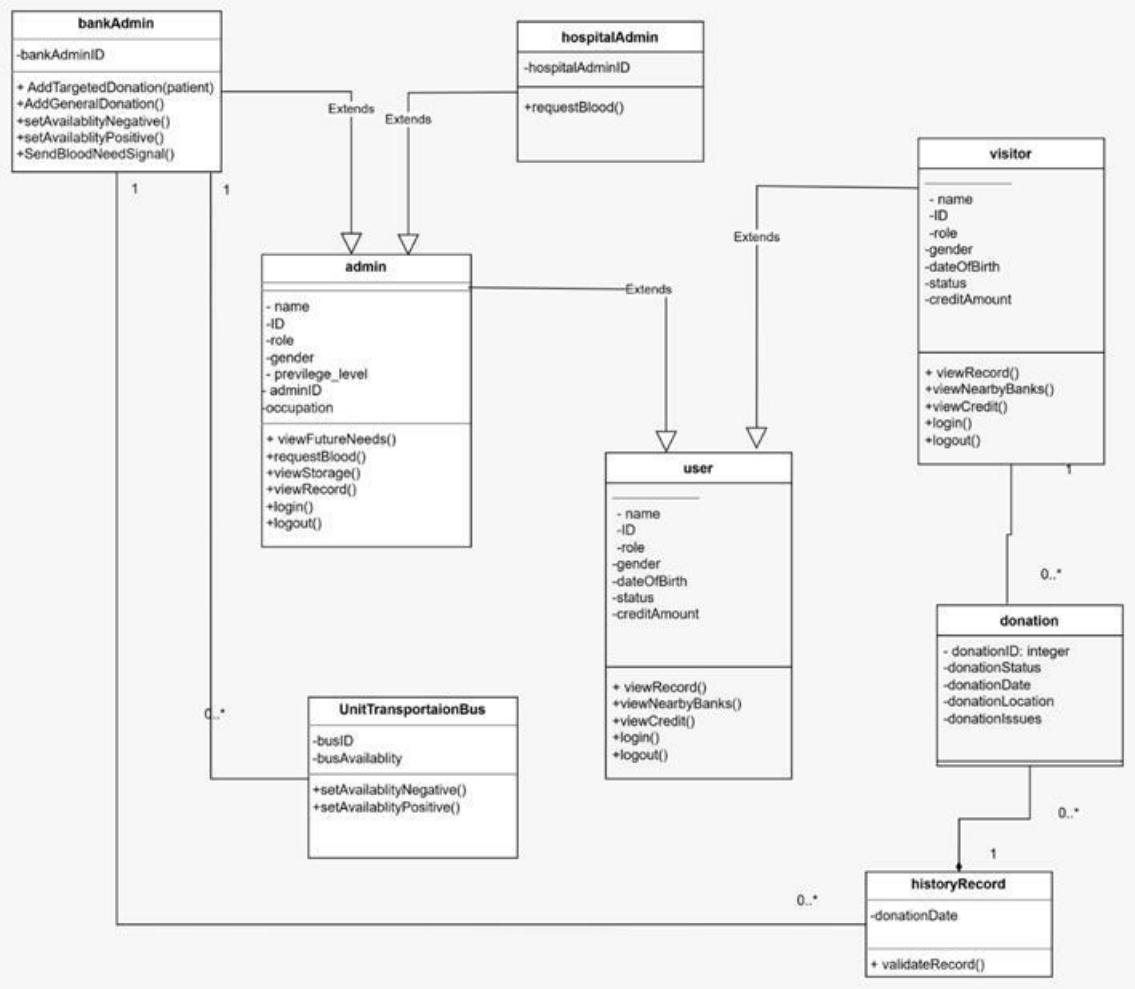

Figure 6: Class diagram

The class diagram above illustrates the main classes the blood bank system uses plus their relationships with each other, the “user” class is the main class since it has the basic functions of authentication and data variables which the classes “visitor” and “admin” inherit, and the “bankAdmin” class possesses most of the system control functions with functions like adding donations to the records of the visitors, adjusting availability of transportation buses and other features, the diagram clarifies the basics of the system and functions within it.

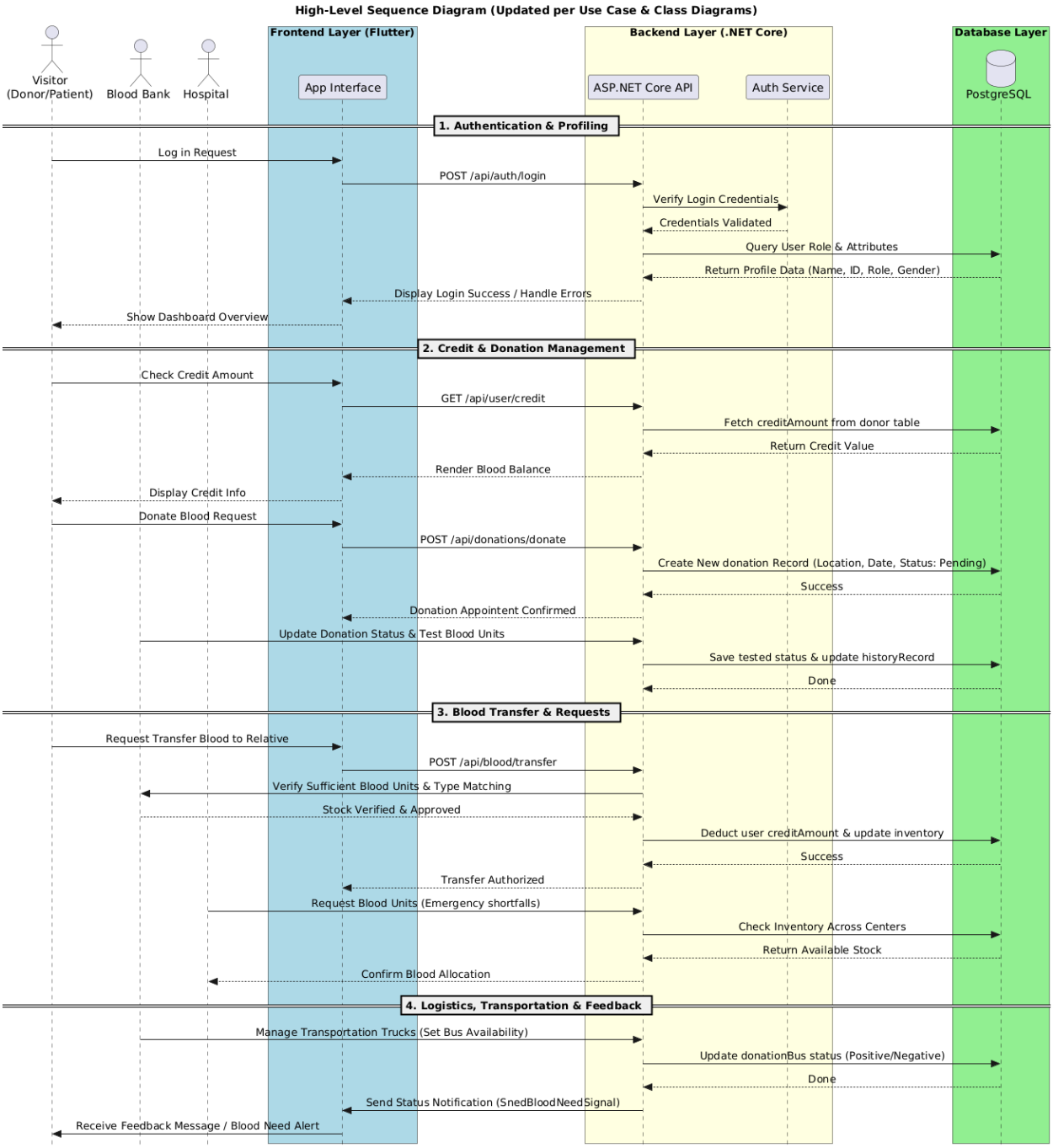

Figure 7: Sequence Diagram

IV. Professional Practice Constraints

The project follows required technical, ethical, social, and healthcare-related constraints to ensure the system is reliable and suitable for use in managing blood donation processes between donors, hospitals, and blood banks.

- Manufacturability Constraints

For the system architecture and interfaces, we will use Flutter for mobile app and website interfaces, ASP.NET for the backend, PostgreSQL for the database, and run the project using Docker. These technologies are selected because of their compatibility with cross-platform development and support for scalable web services.

- Economic Constraints

The project aims to reduce operational costs by minimizing paperwork and manual administrative tasks. Furthermore, the use of digital records and notifications reduces the time and effort required during emergencies. Also, we will use open-source technologies and Docker containers to minimize costs.

- Sustainability

We strive to develop sustainable solutions to avoid harming the environment by reducing the need for paper-based records through digital storage of donor and hospital data. The system can also connect all possible hospitals with the blood bank, further enhancing sustainability.

- Health and Safety Constraints

Health and safety are the most important constraints in our project, as the system will contain accurate and reliable medical information. Therefore, access to patient and donor data will be restricted to authorized personnel only. The system will also be designed to facilitate the provision of necessary blood units in emergencies.

- Ethical Standards Constraint:

We will ensure the system has the highest level of privacy by protecting user information, including donors and patients, and that no one can access it except those authorized.

- Social Values Constraints

Our project aims to support all segments of society by making it easier to register as blood donors and request blood units during emergencies. The system will also contribute to strengthening cooperation between hospitals in different governorates to reduce delays in critical situations.

V. Deliverables

Our blood bank system's main goal is to save time, effort, and potentially manpower by replacing the outdated current paper-based system by making the workflow of storing and retrieving patient/donor information very fast and simple electronically making it almost an automated process

- Doctors from the hospital in need of units will send a simple request using the system rather than give calls and keep waiting for feedback and stay on constant alert.
- Blood bank workers will have a desktop app/website to insert/retrieve information and data to/from for donors and recipients.
- Blood bank workers will have a desktop app/website to manage, update, and keep track of blood unit supply and demand.
- Donors will be able to see the status of their donations clearly and will be provided with notes that explain the reasons for rejection of a donation if it happens.
- Donors and patients will have their records available to check from a simple phone app or website.
- The credit system will encourage people to donate blood since it's risk-free because it tracks how many units you have to your name.
- The credit system will allow donors to transfer their credit to whoever needs it, such as patients.
- The system provides constant overwatch over the blood supply for the bank.

VI. Special features

1. Cross-country donation

A person in need of blood units from outside his area could have his donors donate from their area and transfer their donations to him as a balance/credit.

2. Blood unit transportation trucks

Areas with a large number of donors will have more units to spare, while opposite areas will need more. These trucks take the extra units to the area that requires them, which can be monitored through our system.

3. Supply need prediction algorithm

This algo/AI will have access to the database where it can see the current supply of blood units and also see aspects of donors and recipients (age, history, etc.…) and have the context of the area that it's calculating for, and based on this information, it will predict the required storage needed in the coming time period and predict which types are needed. To be noted, this is an extra feature; we will implement it if we have time.

4. Credit system

Donations with this system will work similarly to how credit works with money: a donor will donate blood units and will have them registered to him as credit. Whenever this credit is required either for self-related or other people's cases, it can be withdrawn from the bank as any blood unit, not necessarily the same unit that was donated.

V. Roles

Regarding distributing the workload across the team, we of course divided it evenly between us. That being said, we agreed to divide the workload as follows:

1. Abdallah and Mohammad are assigned to carry out the backend development using ASP.NET, ensuring the work of all functionalities and classes.

2. Zaid will take an interest in the frontend development using Flutter, making the user interface and user experience simple, beautiful, and easy to deal with.

3. Aws is going to take the database part, managing the queries, user information, and the whole data using PostgreSQL.

VI. Schedule

Time Frame	Task To Be Completed	Status
February-March	Decide project idea	Done
March-April	Look up required tools and technologies to choose from	Done
April-May	Give roles to each team member and practice them	Done
June	Report preparation and presentation	Done
August-September	Get familiarized with tools and libraries	Planning
September-October	Prepare 50% of the front-end interface and setup database	Planning
October-November	Prepare back end to connect front end to the database, add interactivity to the front end	Planning
November-December	Continue the other 50% of the front end and add interactivity to it	Planning
December-January	Test project for bugs and issues, lock in final code	Planning
January	Final report and presentation.	Planning

VII. Bibliography

[1] Flutter Official Documentation

[2] ASP.NET Official Documentation

[3] PostgreSQL Official Documentation

[4] Docker Official Website

[5] draw.io

[6] UML Planet

[7] Blood Bank as explained in Wikipedia

[8] King Abdullah University Hospital

[9] Central Blood Bank for North Region / Irbid

[10] Prince Rashed Bin Al-Hasan Military Hospital